Checked proof simplifications for K215

Normalized source certificates, framed decoder blocks,
and zero-credit enumeration

Prepared for Vladimir Reshetnikov's *ProveIt* repository
21 October 2026

Abstract

We reconcile the current K215 development with all seven additional archives in `docs/incoming/` and implement three further proof components. First, the incoming Lean normalization of a 2,240-entry source certificate is ported to Rocq: a single checked 1,426-entry certificate proves the exact existing source halting equivalence for every natural-valued register file. Second, an arbitrary-tape finite-window theorem and its composition rule are proved in Lean, and applied to 281 literal decoder blocks and the finite conditions of a 234-role decoder product. Third, the concrete Cantor-code tag-lowering argument for ZC215 is formalized in Coq, including the parser, strict decrease, injectivity and every finite scheduling prefix. These extend the incoming work and retain its attribution. We distinguish the smallest incremental machine proof, offered by predecessor-marker variants, from a simpler standalone scheduler, offered by ZC215. A coordinated two-merge/one-selector search tests 457,682,472 complete provider specifications on 18 reference tables. It finds no verified 214-state machine. Proof boundaries, source pins, reproduction commands and verification logs accompany every result.

1 Outcome and scope

The main recommendation is to preserve the exact accepted K215 as the reference endpoint and reduce its proof through small, separately checked interfaces. For another 215-state machine, use a short local equivalence to that endpoint when its changes admit one. For a future independent source proof, ZC215 offers a more substantial semantic simplification: its stack safety follows from an exact injection and needs no initial numerical reserve.

No valid 214-state construction is supplied. The new search extends the incoming one-pair and strict-provider templates, but its failure is a bounded negative result. It is neither a lower bound on machine size nor a reason to conclude that a 214-state machine does not exist.

The repository snapshot is

`53269bcab1efc0f82d3022645bd7cf0c22d84959`

The exact accepted native table has SHA-256

`9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2`

and its dense table has SHA-256

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`

The intended arithmetic specification is

$$\mathrm{ZFC} \vdash [\neg\exists t\ \mathrm{Halt}_M(t) \iff \neg\exists p\ \mathrm{Prf}_{\mathrm{ZFC}}(p, \bot)]\,,$$

where $\mathrm{Halt}_M(t)$ says the blank run reaches halt within t steps. The positive halting/refutation equivalence is the interface to preserve; negation gives the desired consistency statement. The newly checked components below simplify parts of that interface, rather than supplying a new internal arithmetization of the entire proof-assistant development.

The convention is one two-way infinite binary tape, initially zero everywhere, L/R moves only, 215 working states including initialization, and a separate halt excluded from the count. No program tape or external initializer is left uncounted. The repository endpoint is an external proof-assistant theorem equating blank halting with a finite ZFC refutation. Its ordinary consistency calibration establishes the intended ZFC-formalizable equivalence. Neither that endpoint nor this work asserts unconditional nonhalting or supplies a new internal-PA formalization.

Component	New checked result	Scope
Rocq normalization	Exact source halting equivalence, all registers	Source layer
Lean framing	281 blocks restore arbitrary tapes; chains compose	Local decoder
Coq zero credit	Concrete all-natural prefix injection and inequality	Scheduler codec
Coordinated search	Reproducible search and independent controls	No accepted 214

New modules were checked with Lean 4.32.0 and Coq 8.18.0. The latter is the installed checker for the Rocq-language contributions; the production repository's full Rocq lane uses another pinned version. These local checks are not a fresh rebuild of that production endpoint. Standard Lean axiom reports and closed Coq assumption reports are retained.

2 Reconciliation with the current and incoming work

The production proof already contains four simplification phases. Its large Lean dominated-implication source has already decreased from 689,857 to 30,114 bytes, the AX routine from 202,222 to 7,563 bytes, and the project closure from 204 to 185 modules. Offset symmetry has removed sixteen reverse-certificate Lean modules. These are inherited results [2].

The queue README alone does not describe the inspected incoming state. Seven ZIP archives are present at the pinned commit, although the latest prose says that the preceding batch's queue was clear. We inspected the archives themselves. Their names, sizes, Git blob identifiers and hashes are retained in `provenance/incoming.json`. The following identifiers are used here.

Packet	Existing relevant contribution
I1	`K215_Proof_Simplification_2026-10-21 (2).zip`: CM215 Core bridge; actual-checker Rocq symmetry; compact equality; ordinary Lean source lane.
I2	`K215_Proof_Simplification_2026-10-21.zip`: compiled Lean 1,426-entry normalized certificate and short CM215 transport.
I3	`K215_Proof_Transport_2026-10-21.zip`: CM215 transport/Core bridge, graph measurements and bounded searches.
I4	`Structural_Proof_Simplification_2026-10-21.zip`: weighted graph identity and framed decoder products, with formalization still proposed.
I5	`ZFC215_Proof_Simplification_2026-10-21.zip`: ordinary one-sided Rocq source proof; M124 transport in both assistants; source-invariant research.
I6	`ZFC_K215_Proof_Transport_2026-10-21.zip`: CM215 transport and exact proof-data sharing/compression pilots.

Packet	Existing relevant contribution
I7	`zfc-fm215-proof-compression-2026-10-21.zip`: FM215 and its standalone Lean transport, finite decoder certificate and larger bounded local searches.

We do not repeat symmetry, marker designs, the 1,426-entry idea or the 234-role product as new discoveries. The concrete extensions are the Rocq normalization port, the checked arbitrary-tape framing/composition library and its literal blocks, the fully concrete zero-credit proof, and a different coordinated synthesis family. Notices and source pins accompany copied material. Whole copies of the seven reports and compiler/toolchain binaries are excluded.

3 A normalized, one-sided Rocq source proof

3.1 The normalized source certificate now has a Rocq proof

The incoming normalization proposal already reduced the source-correspondence certificate from 2240 entries to 1426, and supplied a Lean proof. The incoming compact-source proposal already supplied the lossless positive-integer path codec and a plain mirror theorem for Rocq. Our contribution combines those two lines of work and fills an explicitly outstanding step: a kernel-checked Rocq theorem for the *same normalized certificate*, using the exact current source programs. Neither the normalization idea nor its Lean implementation is claimed as new.

The resulting theorem is

$$\forall R, \qquad \mathrm{Halts}(p_{215}, (\epsilon, R)) \quad \Longleftrightarrow \quad \mathrm{Halts}(\mathrm{dagPerm}, (\epsilon, R)),$$

where R is an arbitrary natural-valued register file. Both program definitions are byte-for-byte equal to the definitions at the pinned repository revision. The generic checker and the concrete source theorem print *Closed under the global context*. No consistency assumption occurs. The tested adapter exports the existing qualified name `K215F.Source215.p215_dagPerm`; an explicit typed client checks that public name and its original statement.

The mathematical reduction. For finite Boolean paths, put $P \sim Q$ when their infinite extensions by false bits agree at every index. This is exactly the existing `ipeq` relation. The existing `hn_ipeq` theorem preserves halting after a specified number of steps under this equivalence. The new Boolean comparator `ibeq` checks $\sim$: after one path ends, every remaining bit of the other must be false. Its soundness proof is an induction on the paths.

At a stored pair of decrement or halt sites, the normalized checker uses the unchanged `seg` interpreter. Suppose its two successful segments have increment lists L_1, L_2 and actual endpoint paths Q_1, Q_2. The checker verifies

$$\forall r, \quad \mathrm{count}(r, L_1) = \mathrm{count}(r, L_2), \qquad Q_1 \sim P_1, \quad Q_2 \sim P_2,$$

where (P_1, P_2) is the selected successor entry. The inherited segment lemmas produce the actual source and target runs and the pointwise register equations $R_i'(r) = R_i(r) + \mathrm{count}(r, L_i)$. Thus equal registers remain equal. The source endpoint's halting run transfers to P_1 by `hn_ipeq`; the induction hypothesis gives a target run from P_2; the same equivalence theorem transfers that run back to Q_2. Prepending the target segment completes the branch argument.

The induction strictly decreases when the source performs its next decrement test, irrespective of whether either connecting segment has length zero. Both test outcomes are covered by the certificate. A

halt entry terminates immediately. Mirroring exchanges the two programs and the two pointers while preserving both successor indices, so a generic theorem supplies the reverse direction. Only one concrete checker evaluation is required.

What normalization does not authorize. The equivalence identifies only a finite path's *trailing false bits* in the existing mathematical pointer representation. It does not remove arbitrary suffixes, change a consumed leaf path, or delete physical cells from a binary Turing-machine tape. Break and continue still act on the exact consumed path returned by the unchanged interpreter. This distinction is essential: a proof about abstract pointer representations does not itself establish a different tape-frame layout.

Size and data binding. The normalized certificate has 1426 entries: 1425 decrement checkpoints and one halt checkpoint. There are 2851 branch obligations including the initial branch. Removing 814 entries is a 36.3393% reduction. The data plus the retained codec occupy 109958 uncompressed source bytes, compared with 173003 for the incoming compact 2240-entry presentation and 1499170 for the original Rocq offset certificate. The normalized data alone occupy 108633 bytes; the codec occupies 1325 bytes. The new generic checker occupies 193 lines and 8315 bytes, and the concrete source theorem occupies 21 lines and 757 bytes. These figures measure source representation, not total proof-object size or verification speed.

The two proof assistants share the exact JSON certificate with SHA-256

`6d30d89253629525b0af3066c7fec67b052c07413938fc7a94dd96ab1ff406e2`

The incoming JSON writes pointer bits most-significant-first; the retained Rocq codec recursively stores the head bit in the least-significant constructor. The renderer performs this explicit conversion. An independent binding checker compares all decoded paths and both successors in every row. This audit is separate from the Rocq source theorem: even an incorrect producer cannot make an invalid source correspondence pass the proved checker. The positive-integer codec has no fixed decoding-fuel limit; the longest 176-bit pointer is represented without truncation.

Verification boundary and integration. The focused build compiles all twelve project modules needed for the new source theorem, then the public adapter and its typed client. Its receipts record commands, exact source hashes, raw output hashes, per-module times, and peak resident memory. The toolchain is Coq 8.18.0; the inherited, proved `Compat818` module adapts standard-library lemma names in the private copy of `RMEquiv`. No axiom or executable-definition change is introduced. The original Rocq 9.1 dependency source is retained unchanged for that integration route. All twelve source modules compiled in 80.632 seconds of summed compiler wall time, with peak memory 824648 KiB. A separate recursive `coqchk` of the concrete source theorem completed successfully in 521.235 seconds, with exit code zero. The adapter and typed client took a further 4.478 seconds. The source module compiled in 13.718 seconds in the recorded run; its forward VM reduction reported 6.874 seconds. These are observations, not a controlled speed comparison with a baseline.

The new source theorem is a replacement component of the established machine proof. It does not by itself constitute a fresh rebuild of the complete 237-module ZFC endpoint. The final integration step is to retain the public source facade, update its generator and dependency manifests, rebuild `Correct215` and `CorrectDense215`, and inspect their endpoint assumption reports. The binary table and the logical calibration remain the ones already present in the repository.

4 Kernel-checked framing for the decoder product

4.1 Preserve the physical context

The source has 4,751 primitive occurrences and 4,750 internal tree occurrences. Repeated subtrees alone do not justify sharing their decoder proofs: entry control, nearby pointer bits and contextual returns matter. I4 and I7 instead use roles

$$(\text{source node},\ \text{physical control},\ \text{last at most two pointer bits}).$$

For exact K215 this finite product has 234 roles, 371 internal edges and 97 primitive-entry edges. Internal edges use 281 distinct minimal local windows. Their widths one, two and three have counts 252, 25 and 4. The longest local execution takes seven binary steps. These data and counts are inherited; the new contribution is their kernel-checked framing infrastructure and concrete instantiation.

4.2 A finite partial tape as the interface

A local certificate supplies known cells, an entry control, current head, duration and expected exit. Its evaluator rejects before reading a cell not represented in the finite list. It uses the literal table, without a register macro oracle.

For a partial tape C and arbitrary complete tape $t : \mathbb{Z} \to \{0, 1\}$, let $C \oplus t$ agree with the first matching entry of C where one exists, and with t elsewhere. Local writes update the same first matching entry. The elementary write lemma is

$$(\mathrm{put}(C, i, b)) \oplus t = \mathrm{write}(C \oplus t, i, b)$$

when i is represented in C. It holds for arbitrary finite lists; duplicate generic entries are harmless under first-match semantics. Concrete windows have distinct consecutive coordinates.

Theorem 4.1 (Framed block soundness)**.** *Suppose the finite checker accepts an n-step block from q to q', with $n > 0$, unchanged local cell list, and head displacement from 0 to 1. At every integer location a, for every complete tape matching the specified window there, the actual machine executes exactly n steps from (q, a, t) to $(q', a + 1, t)$. The entire tape, including its arbitrary exterior, is restored.*

Proof. The write lemma makes each successful local step refine one full-tape step. Induct on the exact duration. An unknown read is rejected, so the refinement never depends on unspecified exterior contents. Equality of final and initial cell lists restores the overlay. Translation commutes with full machine steps and finite execution; translate the result from coordinate zero to arbitrary a. □

The generic `Chain` relation concatenates certified blocks. It records membership in the accepted list and the explicit window premise at each successive location. The checked `chain_sound` theorem composes the results by the run-addition law. It quantifies arbitrary finite chains and full tapes; it does not infer missing window facts from execution samples.

4.3 Concrete statements and independent controls

The supplied Lean development checks the literal table's 215 working controls, all 281 blocks including positive duration, and the finite structural conditions for all 234 roles. These include child and target references, decreasing source-node indices, suffix updates, matching block entry/exit controls and literal

primitive-entry actions. It derives the arbitrary-tape block theorem and the finite-chain theorem for this concrete data.

The block acceptance theorem reports no axioms. For semantic results, the reported axioms are `propext`, `Classical.choice` and `Quot.sound`. No theorem contains `sorryAx` or uses `native_decide`. A fresh three-module serial build imports no project objects. It took 41.646 compiler seconds in this run, including 38.495 for the concrete data module. These are absolute observations, not a speed comparison or a full endpoint timing.

An independent checker, importing none of the generator, matches all 430 native actions and every generated node and role. It verifies expansion to the complete pinned 4,751-site list, root reachability of all roles and every local trace. Four malformed families were tested against all 281 blocks: deleting the leftmost cell, changing the exit, shortening duration by one and setting duration to zero. All reject. The 29 nonempty truncated windows fail by reading an unspecified exterior cell. Seven malformed structural examples also reject.

4.4 The remaining integration boundary

Structural validity alone does not validate execution or positive duration; it must be combined with block acceptance. It also does not give semantics to primitive opcodes or arguments. An independent control confirms that changing a leaf argument can leave structural validity intact. That is expected: primitive semantics belongs to another interface. Duplicate roles need not invalidate the local theorem either, although the concrete product has no such redundancy.

Still required are path-to-window/`Chain` witnesses for the actual source, an in-kernel identity with upstream source data, and composition with existing primitive, register-box, pointer-return and coverage theorems. The complete expansion binding here is independently checked data outside Lean's kernel. This does not replace `KSites215`, verify full runtime through the product, or rebuild the ZFC endpoint. It completes a specific missing local foundation of the incoming proposal and supplies a concrete next interface.

5 A concrete zero-credit theorem for ZC215

5.1 Include the actual command codec

ZC215 exhaustively schedules raw candidates $0, 1, 2, \ldots$ with DROP tag zero and NIP tag one. Both consume one Cantor field. Its older generic prototype assumed a lowering map and its main properties. The new `TagLowering.v` constructs those objects for the actual codec and proves the properties.

Use Cantor pairing

$$P(a, b) = \frac{(a + b)(a + b + 1)}{2} + b.$$

Fields are [0] for DROP, [1] for NIP, $[2, x]$ for $\mathsf{MP}(x)$, and $[c + 3, y, x]$ for $\mathsf{AX}(c; x, y)$. Fields are nested with P. A final command has continuation residual zero; a nonempty remaining word has residual one plus its raw code.

That distinction handles the zero alias: the one-command word [DROP] has raw code zero, but a final DROP inside a nonempty continuation must be represented through residual one. Encoding injectivity applies to nonempty words. The empty word is not silently identified with candidate zero.

The file proves the recursive triangular-number definition equals the displayed formula, gives an executable parser, and proves both inverse laws between natural raw codes and nonempty command words. Input

n needs $n+1$ fuel units: the next nonempty continuation's raw index is strictly smaller than n. This is an all-natural proof, not bounded parser testing.

5.2 An injective matching to earlier DROPs

Let W_n be the unique word parsed from n. For a NIP occurrence at position j of W_n, replace it by DROP and let the resulting raw index be m. Put $F(n,j) = (m,j)$. At the modified field,

$$P(1,r) - P(0,r) = r + 1 > 0.$$

Enclosing pairings and positive continuation constructors preserve the strict inequality, so $m < n$. The map is injective: replacing the output's DROP at its recorded position j by NIP reconstructs the original word and index.

Theorem 5.1 (Concrete prefix credit)**.** *For every prefix consisting of all $W_0, \ldots, W_{n-1}$ followed by the first j commands of W_n, one has $\#\mathsf{NIP} \le \#\mathsf{DROP}$. If position j is NIP, then already before executing it one has $\#\mathsf{NIP} + 1 \le \#\mathsf{DROP}$.*

Proof. Map every NIP occurrence in the prefix through F. Each image is a DROP in a strictly earlier completed candidate, hence lies in the prefix. Injectivity gives the first inequality. Apply the same map to the existing NIPs together with the imminent NIP for the second inequality. Its image is also in an earlier completed candidate. □

The public statements are

```
zero_credit n j :
  length (pops n j) <= length (pushes n j).
positive_credit_before_nip n j : Pop (n,j) ->
  S (length (pops n j)) <= length (pushes n j).
```

These have no candidate-specific premises. The 393-line, 14,514-byte file uses the standard library. Fresh compilation and recursive `coqchk` pass; five printed endpoint assumption reports are closed. Recorded times are 4.706 and 77.657 seconds, respectively.

5.3 What disappears from source safety

A protected-stack invariant can begin with zero theorem heads above an arbitrary opaque tail J. DROP adds a head, NIP always finds one, and MP/AX need only preserve that lower bound. The proof assigns no positive credit to AX and does not require an initial tail such as $J = 5$ to decode as a list of theorems. It removes the reserve/debt estimate and theorem-valued-tail startup obligation.

It does not remove theoremhood requirements for top/cache when used by commands, source completeness, exhaustive scheduling or binary frame validity. The parser and count model still need an adapter to the actual source and physical stack. Thus this is a new concrete scheduler-codec theorem, not a new exact-table ZC215 consistency endpoint.

6 Which other 215-state candidate is easiest?

6.1 The incremental objective

After importing K215's endpoint, the clearest smallest local argument is the two-marker M124/FM215 family. Each changes four native rows, merges old roles 124 and 127 at 124, and reuses 127 as selector. The unique incoming old rows are

$$104/1 = 1R124, \qquad 110/0 = 0R127.$$

Therefore every run from control zero, on every initial tape, satisfies

$$q = 124 \Rightarrow t(h-1) = 1, \qquad q = 127 \Rightarrow t(h-1) = 0.$$

These are immediate inductive facts. At read one the roles agree. At read zero the selector reads the predecessor bit and uses a three-step block with the same final head and whole tape as the old one-step action. Other transitions transfer in one step. Positive stuttering gives halting equivalence without rerunning calculus calibration, primitive coverage or blank initialization.

Those proofs already exist in the incoming work: I5 supplies standalone M124 transport in both assistants; I7 supplies FM215 Lean transport. We credit them as existing contributions. Their remaining tape-model/endpoint adapters are focused integration tasks. CM215 needs a three-clause invariant, but several incoming packets already check its actual Core/dense bridge. It may therefore be the quickest practical integration despite its slightly larger local proof. A short incremental proof still inherits the complete K215 foundation.

6.2 M124 and FM215 are different machines

An independent comparison identifies exactly one different native row:

Row	M124	FM215
127/1	$1R59$	$1R109$

Both providers' zero-rows are $0R62$, so either works in the intended simulation. Their one-rows differ: $59/1 = 1R72$, whereas $109/1 = 1R42$. They share a construction pattern, but their rooted transition graphs are nonisomorphic even after global left/right reflection. The comparison and exact tables are included; M124 is also bound to the current DR215 marker-family table. Similar proof structure must not be confused with literal machine identity.

6.3 The independent source objective

For a future fresh source proof, ZC215 offers the best demonstrated simplification: the injection removes an entire safety calculation. GC-ZC215 also simplifies generalization using a command tag as binder, but changes more source and physical interfaces. Direct-transfer ST215 reduces some primitive code while retaining other compiler/stack obligations. Forced-transfer FT215 changes little of the table yet has a debt-seven scheduler argument.

Edited-row count, primitive count and source-DAG count are therefore poor surrogates for proof difficulty. No measured comparison establishes that an alternative has a smaller complete dependency closure than current K215. The recommendation depends on whether one wants cheap relative certification, a cleaner independent scheduler, or a physical state reduction.

7 A coordinated search for 214 states

7.1 Two mergers sharing one selector

A single merger often needs the recovered state to remember the old origin. The new search attempts two disjoint pair mergers with one common selector. Success would give $215 - 2 + 1 = 214$ working controls.

At the exceptional branch of the first retained control, temporarily write tag zero in the scanned cell; at the second, write tag one, and also test the opposite assignment. Move to an adjacent origin-marker cell and enter the shared selector. It preserves the marker, returns to the temporary tag and chooses one of two existing providers. The provider must restore the correct old tape effect and join a positive old execution at corresponding control and identical head and tape.

Both pairs may conflict on read zero, or both on read one: the temporary tag identifies the pair. This extends the earlier opposite-read template. All four synthesized rows are active simultaneously and paths may revisit retained controls or the selector, extending strict-provider screens. Existing provider rows themselves are not arbitrarily resynthesized.

7.2 Sound finite-window screening

For each input, an inductive radius-five domain is seeded by its actual blank initial window. Radius five means eleven cells. Every abstract shift admits both possible entering exterior bits. A separate list-based checker verifies initialization and closure independently of the bit-mask worklist generator. An absent window is unreachable; an admitted one need not be concretely reachable.

For a trial, each old and new path is capped at 64 literal steps and may not read outside the window. Successful obligations would give positive old and new blocks with exact corresponding endpoints. Positive lengths, terminal agreement and determinism then give blank halting equivalence. Unchanged rows transfer in one step. Window preservation frames the arbitrary exterior.

A failed join, a window escape or a bound exit is a failed certificate search, not necessarily a concrete counterexample. A long matching blank trace is not an unbounded proof either. Rejected trials are not presented as accepted 214-state candidates.

7.3 Results and controls

Seed	Families	Provider specifications	Accepted
Primary K215	500	22,684,500	0
GC-ZC215	690	31,304,610	0
Context returns	734	33,300,846	0
Shallow context returns	788	35,750,772	0
Incoming FM215	456	20,688,264	0
00_commands120	592	26,858,448	0
01_composed	500	22,684,500	0
02_composed	500	22,684,500	0
03_conditional-marker	458	20,779,002	0
04_direct-transfer	500	22,684,500	0
05_mp-first-transfer	546	24,771,474	0
06_rebalanced	456	20,688,264	0
07_reencoded	540	24,499,260	0
08_rho6	688	31,213,872	0
09_rho6	638	28,945,422	0
10_zero-spare	500	22,684,500	0
11_zero-spare-mp-first	546	24,771,474	0
m124	456	20,688,264	0
Total	10,088	457,682,472	0

These are complete two-provider specifications per input, not individual rows or necessarily distinct transition graphs. There are 213 retained old controls after two deletions, hence 213^2 provider assignments per pair/tag family. Direct provider targets exclude halt and the new helper, although paths can reach halt or revisit the helper. Start is excluded from the four merged origins. Different allocations are retained as distinct search inputs; no claim of 18 pairwise nonisomorphic machines is needed.

All eighteen blank-seeded domains include both read symbols at every working state. A synthetic eleven-to-ten-state gadget has both pairs conflicting on read zero; the search finds the intended temporary-tag solution. An independent dictionary-tape simulator checks all 32 local contexts. A wrong selector target fails 16, as does erasing the temporary tag. This tests the additional mechanism and tape restoration. It is a local control, not a ZFC machine.

All searches completed. The core five were replayed with final C++ source; the thirteen extensions were fresh complete executions of that same compiled source. A final audit regenerated every invariant byte-for-byte and independently checked all eighteen. These receipt scopes are distinguished rather than labeling the final identity audit as a second execution of every search. The result is negative within this family. Source redesign, changed provider rows, other invariants, nonlocal joins and larger coordinated mergers remain outside it.

8 Integration plan and trust boundaries

1. **Adopt normalized source correspondence first.** Preserve the exact programs and public statement. Replay the checker, certificate and facade under the production Rocq toolchain; rebuild affected endpoints and inspect assumptions. Update the generator alongside its output.

2. **Finish one alternate-machine adapter.** CM215's existing Core bridge minimizes remaining

integration effort; M124/FM215 minimizes the local invariant. Bind the exact table before composing the accepted ZFC endpoint.

3. **Attach framing to the source relation.** Prove path-to-window facts and primitive/return interfaces, then replace occurrence checks only after full site coverage compiles. Keep all context required by the physical decoder.

4. **Attach zero credit to the actual ZC215 schedule.** Connect refill and parsing to $W_0, W_1, \ldots$ and occurrence counts to protected-stack semantics. Keep calibration and compiler completeness as explicit shared obligations.

5. **Measure the complete result.** Use the same compiler versions, clean or hash-bound dependencies and endpoint clients. Source bytes, object bytes, compiler time, kernel time and peak memory should be reported separately.

A source equivalence is not a full binary-to-ZFC check. A whole-tape theorem in one model is not an imported endpoint in another. A finite graph does not establish the semantics of its primitive labels. The packet makes these boundaries explicit so that shortening the proof does not weaken the statement or introduce an unproved premise.

9 Further research questions

1. **Can correspondence data disappear entirely?** I4's normalized weighted test graphs agree by checkpoint ordinal. A generic adequacy theorem could replace the 1,426-entry relation by graph equality. It must handle zero padding, exact contextual break/continue targets and positive progress.

2. **Can the 97 primitive entries share return contracts?** Determine a sufficient finite context for pointer carry, strip and return operations. Equal primitive labels alone do not imply identical continuations.

3. **Can provider resynthesis enable selector sharing?** Synthesize a few provider edits and verify all their old uses simultaneously. Exact tape restoration and positive joins, rather than sampled execution, are the contract.

4. **Can zero credit enable a smaller physical initializer?** Opaque tails and zero reserve are now justified for the concrete codec. Use this freedom while retaining theorem-valued top/cache, exhaustive enumeration and valid frames. Count physical states after allocation, not only source DAG nodes.

5. **Can more than two pairs share history storage?** A temporary bit stores one distinction. Nearby bits or a reusable short scan might encode more roles, but must be restored without breaking provider uses.

6. **Which architecture wins a controlled full build?** Compare normalized certificates, graph adequacy and framed products at the same endpoint. Smaller source data can still increase normalization or dependency cost.

10 Artifacts and reproduction

The archive contains the complete LaTeX/PDF article, a top-level verifier, provenance inventory, and four focused components:

- `normalized_rocq/`: Exact programs, normalized certificate, ordinary checker, facade, dependency sources, notices, build runner and verification logs.
- `decoder_lean/`: Framed-window and chain proofs, finite checker, literal K215 data, frozen inputs, independent audit and fresh serial build driver.
- `zero_credit/`: Concrete codec/prefix proof, standalone Coq runner, compiler/`coqchk` logs and M124/FM215 comparison.
- `search214/`: Coordinated C++17 search, compressed invariants, eighteen reference tables, independent validators, controls and exact receipts.

Typical commands, using fresh work directories, are

```
python3 verify.py
python3 zero_credit/reproduce.py --output /tmp/zc-credit-fresh
python3 decoder_lean/build.py --lean /path/to/lean --out /tmp/decoder-fresh
python3 search214/verify.py --work /tmp/search-audit
python3 search214/verify.py --work /tmp/search-replay --search
```

The normalized-source README gives its exact namespace/compatibility commands. Top-level verification checks released hashes and finite bindings; it does not silently invoke or claim a kernel proof. Compiler objects, toolchains and caches are excluded. Rebuild the PDF with two `pdflatex` passes. The numerical calculus, baseline compiler, parent tables and inherited verification libraries remain contributions of their original authors; component notices preserve that attribution.

References

[1] Vladimir Reshetnikov and contributors. *ProveIt, Busy Beaver ZFC research*, pinned revision `53269bcab1efc0f82d3022645bd7cf0c22d84959`. https://github.com/VladimirReshetnikov/ProveIt/tree/53269bcab1efc0f82d3022645bd7cf0c22d84959/Computability/BusyBeaver/Research/ZFC.

[2] ProveIt contributors. *Shorter checked K215 proofs*, phases 1–4, exact endpoints and raw records. https://github.com/VladimirReshetnikov/ProveIt/tree/53269bcab1efc0f82d3022645bd7cf0c22d84959/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification.

[3] ProveIt incoming contributors. *Seven proof-simplification and transport archives*, I1–I7, with exact identities in the accompanying inventory. https://github.com/VladimirReshetnikov/ProveIt/tree/53269bcab1efc0f82d3022645bd7cf0c22d84959/docs/incoming.

[4] ZC215 contributors. *Zero-credit enumeration*, ordinary tag-lowering proof and calibration. https://github.com/VladimirReshetnikov/ProveIt/tree/53269bcab1efc0f82d3022645bd7cf0c22d84959/Computability/BusyBeaver/Research/ZFC/candidate-k215/alternatives/zero-credit.